Chapter 9 | Exercise solutions with Python

Scaling Data, Compute, and Expert Capacity

This notebook solves Exercises 5–9 and 11. It compares model sizes at a fixed compute budget, checks the stability of routing probabilities, follows requests that exceed expert capacity, and derives the optimum of the chapter's toy loss. Exercises 1–4, 10, and 12 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch09_scaling" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Refine the sweep

Imagine choosing between a model with 10 billion parameters and another with 13 billion. At the same training budget, the second model spends more on every token and can therefore process fewer tokens. The exercise asks whether the improvement in the chapter's size penalty is large enough to offset the increase in its token penalty. The chapter table tried sizes 5, 10, 20, 40, and 80 billion and found the lowest loss at 10 billion. Now test more choices between 10 and 20.

The notation $N_B = N/10^9$ and $D_B = D/10^9$ expresses the parameter count and token presentations in billions. Substituting the raw counts $N = N_B 10^9$ and $D = D_B 10^9$ into $C \approx 6ND$ gives $C \approx 6 \cdot 10^{18} N_B D_B$. The same budget fixes their product.

$$N_B D_B = \frac{2.4 \cdot 10^{22}}{6 \cdot 10^{18}} = 4000, \qquad D_B = \frac{4000}{N_B}.$$

For each size, `allocation_sweep` solves for its token allocation and evaluates the toy loss $1 + 0.4/\sqrt{N_B} + 2/\sqrt{D_B}$. Each returned row contains model size, token count, and loss, in that order. We select the minimum by the loss in the third field and use the full precision of the returned values. The table formatting only changes how the numbers appear.

```
from examples.ch09_scaling.model import (
    BILLION, allocation_sweep, training_flops,
)

sweep_budget = 2.4e22
chapter_sizes = (5, 10, 20, 40, 80)
refined_sizes = (10, 12, 13, 14, 20)
chapter_rows = allocation_sweep(sweep_budget, chapter_sizes)
refined_rows = allocation_sweep(sweep_budget, refined_sizes)

chapter_best = min(chapter_rows, key=lambda row: row[2])
refined_best = min(refined_rows, key=lambda row: row[2])
assert chapter_best[0] == 10
assert refined_best[0] == 13

print("Model size (B)   Token count (B)      Toy loss")
for row in refined_rows:
    n_b, d_b, loss = row
    mark = "  <- best" if row == refined_best else ""
    print(f"{n_b:>14g}   {d_b:>15.6f}   {loss:>11.9f}{mark}")
print(f"Chapter table's best loss: {chapter_best[2]:.9f}")
```

```
OUTPUT
Model size (B)   Token count (B)      Toy loss
            10        400.000000   1.226491106
            12        333.333333   1.225014565
            13        307.692308   1.224957582  <- best
            14        285.714286   1.225226092
            20        200.000000   1.230864075
Chapter table's best loss: 1.226491106
```

The best refined row gives 13 billion parameters, approximately 307.692308 billion token presentations, and a toy loss of about 1.224957582. The 10-billion row still has its previous loss of about 1.226491106. We added the 12-, 13-, and 14-billion choices between the original 10- and 20-billion rows, so the sweep could now select an allocation that the chapter's table did not test.

We can see the tradeoff in the two loss terms. From 10 to 13 billion parameters, the size penalty falls from about 0.126491 to 0.110940. The token penalty rises from 0.100000 to about 0.114018. The decrease in the first term is larger than the increase in the second, so the total falls.

To verify the budget, use the returned token allocation with its full precision, rather than the rounded number printed in the table. Multiplying each count by 10^9 converts it to the units expected by `training_flops`. The helper can use a fractional token count for this calculation because solving the budget equation is a continuous approximation. A real run would choose a whole number of token presentations.

```
best_size_b, best_tokens_b, _ = refined_best
best_parameters = best_size_b * BILLION
best_tokens = best_tokens_b * BILLION
estimated_flops = training_flops(best_parameters, best_tokens)

np.testing.assert_allclose(
    estimated_flops, sweep_budget, rtol=1e-14, atol=0,
)

print(f"Parameters as a raw count: {best_parameters:,}")
print(f"Token presentations as a raw count: {best_tokens:,.4f}")
print(f"Estimated FLOPs: {estimated_flops:.9e}")
```

OUTPUT

```
Parameters as a raw count: 13,000,000,000
Token presentations as a raw count: 307,692,307,692.3077
Estimated FLOPs: 2.400000000e+22
```

The check agrees at the scale of the budget, with relative tolerance 10^{-14} and zero absolute tolerance. A budget of $2.4 \cdot 10^{22}$ is too large for a useful comparison based on a small absolute error. The relative tolerance compares the discrepancy to the magnitude of the budget.

The chosen row minimizes the chapter's made-up loss over the tested sizes. Its advantage comes from changing which sizes are available at the same compute budget. Exercise 11 finds the minimum of this same curve when any positive size is allowed.

6. Check the score shift

A router looks at a row of scores, called **logits**, and requests the expert with the largest score. For this token the four expert probabilities are $(0.1, 0.6, 0.2, 0.1)$. Take their natural logarithms to build a single logit row. Applying softmax to those logs exponentiates them back to the four probabilities, whose sum is already one.

The shape $[1, 4]$ means one token and four experts. Adding a common constant c to all four scores preserves their order. In particular, the second score stays the largest. The output ID is 1 because Python indices start at zero. The expert is Expert 2 in the chapter's one-based numbering.

The probabilities should also agree. To see why, write s_i for the logit of expert i. The shift contributes the same multiplicative factor $exp(c)$ to the numerator and denominator.

$$\frac{\exp(s_i + c)}{\sum_j \exp(s_j + c)} = \frac{\exp(c)\exp(s_i)}{\exp(c)\sum_j \exp(s_j)} = \frac{\exp(s_i)}{\sum_j \exp(s_j)}.$$

That factor cancels in exact arithmetic. In Python, exponentiating a value near 1,000 directly would overflow in float64. The chapter's softmax first subtracts the largest logit in the row, leaving nonpositive inputs to the exponential and at least one exponential of zero. The subtraction also cancels a common shift in exact arithmetic. Small rounding differences may remain because the additions and subtractions use finite precision.

```
from examples.ch09_scaling.model import route_top1

target_probabilities = np.array(
    [[0.1, 0.6, 0.2, 0.1]], dtype=np.float64
)
base_scores = np.log(target_probabilities)
shifted_scores = base_scores + 1_000
base_prob, base_ids, base_gate, base_accept = route_top1(
    base_scores, capacity=1
)
shifted_prob, shifted_ids, shifted_gate, shifted_accept = route_top1(
    shifted_scores, capacity=1
)

probability_atol = 1e-12
np.testing.assert_array_equal(base_ids, [1])
np.testing.assert_array_equal(shifted_ids, base_ids)
np.testing.assert_array_equal(base_accept, [True])
np.testing.assert_array_equal(shifted_accept, base_accept)
for actual, expected in (
    (base_prob, target_probabilities),
    (shifted_prob, base_prob),
    (base_gate, [0.6]),
    (shifted_gate, base_gate),
):
    np.testing.assert_allclose(
        actual, expected, rtol=0, atol=probability_atol
    )

with np.printoptions(precision=12):
    print("Original logits")
    print(base_scores)
    print("Shifted logits")
    print(shifted_scores)
    print("Original probabilities", base_prob)
    print("Shifted probabilities ", shifted_prob)
    print("Original ID, gate, acceptance", base_ids,
          base_gate, base_accept)
    print("Shifted ID, gate, acceptance ", shifted_ids,
          shifted_gate, shifted_accept)
prob_delta = np.abs(base_prob - shifted_prob).max()
gate_delta = np.abs(base_gate - shifted_gate).max()
print(f"Max probability difference {prob_delta:.3e}")
print(f"Max gate difference        {gate_delta:.3e}")
```

OUTPUT

```
Original logits
[[-2.302585092994 -0.510825623766 -1.609437912434 -2.302585092994]]
Shifted logits
[[997.697414907006 999.489174376234 998.390562087566 997.697414907006]]
Original probabilities [[0.1 0.6 0.2 0.1]]
Shifted probabilities  [[0.1 0.6 0.2 0.1]]
Original ID, gate, acceptance [1] [0.6] [ True]
Shifted ID, gate, acceptance  [1] [0.6] [ True]
Max probability difference 8.743e-15
Max gate difference        6.550e-15
```

The selected ID is exactly the same in both runs. One request fits within capacity one. The gate is the probability of the selected expert, about 0.6, so it agrees as well. The helper keeps that probability as the gate after choosing one expert. It does not rescale it to one. The code checks the IDs and Boolean masks for exact equality and uses absolute tolerance 10^{-12} with zero relative tolerance for float64 probabilities and gates. The last two lines show the rounding differences before the arrays are formatted for display.

7. Move the capacity limit

Put score 2 in one column per row and leave all the other scores at zero. Because 2 is strictly greater than zero, every row has a unique winner. The eight rows request experts with zero-based IDs $0, 1, 1, 1, 1, 1, 2, 3$. Written with the chapter's expert labels, the request counts are $(1, 5, 1, 1)$ for Experts 1 through 4.

Capacity is a separate limit on how many requests each expert accepts. The teaching router gathers the requests for one expert in original row order and keeps the first `capacity` of them. Expert 2 receives rows 2 through 6, so at capacity 2 it accepts rows 2 and 3. The other experts receive one request each and can accept all of theirs. These row numbers start at one to match the book.

```
winning_ids = np.array([0, 1, 1, 1, 1, 1, 2, 3])
num_rows, num_experts = 8, 4
capacity_scores = np.zeros((num_rows, num_experts))
capacity_scores[np.arange(num_rows), winning_ids] = 2

print("Score matrix, one row per token")
print(capacity_scores)
print("Requested expert labels", (winning_ids + 1).tolist())
request_counts = np.bincount(winning_ids, minlength=num_experts)
print("Requests per expert    ", request_counts.tolist())
```

```
OUTPUT
Score matrix, one row per token
[[2. 0. 0. 0.]
 [0. 2. 0. 0.]
 [0. 2. 0. 0.]
 [0. 2. 0. 0.]
 [0. 2. 0. 0.]
 [0. 2. 0. 0.]
 [0. 0. 2. 0.]
 [0. 0. 0. 2.]]
Requested expert labels [1, 2, 2, 2, 2, 2, 3, 4]
Requests per expert     [1, 5, 1, 1]
```

For each capacity, use the Boolean acceptance mask to select the IDs of accepted rows. Count those IDs with `np.bincount`. Setting `minlength=4` keeps one slot per expert even if an expert accepted no rows. To find overflow rows, locate the false entries in the mask and add one to the resulting indices. A row that overflows still has a selected ID.

```
expected_overflow = {2: [4, 5, 6], 3: [5, 6], 5: []}

for capacity in (2, 3, 5):
    _, requested_ids, _, accepted_rows = route_top1(
        capacity_scores, capacity=capacity
    )
    accepted_counts = np.bincount(
        requested_ids[accepted_rows], minlength=num_experts
    )
    overflow_rows = np.flatnonzero(~accepted_rows) + 1
    np.testing.assert_array_equal(requested_ids, winning_ids)
    np.testing.assert_array_equal(
        accepted_counts, np.minimum(request_counts, capacity)
    )
    np.testing.assert_array_equal(
        overflow_rows, expected_overflow[capacity]
    )
    print(f"Capacity {capacity}")
    print("  Accepted per expert", accepted_counts.tolist())
    print("  Total accepted     ", int(accepted_rows.sum()))
    print("  Overflow rows      ", overflow_rows.tolist())
```

OUTPUT

```
Capacity 2
  Accepted per expert [1, 2, 1, 1]
  Total accepted      5
  Overflow rows       [4, 5, 6]
Capacity 3
  Accepted per expert [1, 3, 1, 1]
  Total accepted      6
  Overflow rows       [5, 6]
Capacity 5
  Accepted per expert [1, 5, 1, 1]
  Total accepted      8
  Overflow rows       []
```

At capacity 3, the extra slot at Expert 2 admits row 4, leaving rows 5 and 6 in overflow. At capacity 5 all eight requests fit. In all three runs the three other experts accept one row apiece.

Empty slots elsewhere cannot take Expert 2's extra requests. For example, at capacity 2 there are eight slots across the four experts, but Expert 2 has only two of them. Each of the other three experts has one unused slot, which this router cannot give to Expert 2. There is no retry with another expert after overflow. Changing capacity therefore changes which requests are executed while leaving the score matrix and the requested expert for each row the same.

8. Check the residual

The chapter's `teaching_weights()` gives the router matrix and both expert projections. The input $x = (1, 0, 0, 0)$ is a row of shape $[1, 4]$. Multiplying it by the router matrix selects the matrix's first row, because the first input coordinate is 1 and all the rest are zero. That row contains the same logarithms as in Exercise 6, so the router requests Expert 2 with gate 0.6.

At capacity one, the expert's up projection copies the input's first coordinate into its first hidden unit. The remaining hidden coordinates are zero. ReLU preserves this vector by replacing only negative entries with zero. The down projection uses the first row of the expert's down matrix to produce $(2, -1, 0, 3)$. Multiplying by the gate gives the branch update.

$$0.6(2, -1, 0, 3) = (1.2, -0.6, 0, 1.8).$$

Adding the original input then gives $(2.2, -0.6, 0, 1.8)$. At capacity zero, the route still requests Expert 2 and the gate still records 0.6. The acceptance mask is false. The branch helper starts with a zero update and fills it only for accepted rows. So this row keeps its zero update and the residual addition returns $(1, 0, 0, 0)$.

```
from examples.ch09_scaling.model import expert_branch
from examples.ch09_scaling.run_examples import teaching_weights

bypass_router, bypass_up, bypass_down = teaching_weights()
bypass_input = np.array([[1., 0., 0., 0.]], dtype=np.float64)
bypass_scores = bypass_input @ bypass_router

print("Router, up, and down shapes",
      bypass_router.shape, bypass_up.shape, bypass_down.shape)
with np.printoptions(precision=9):
    print("Input row          ", bypass_input)
    print("Computed router row", bypass_scores)
    print("First row of router", bypass_router[:1])
np.testing.assert_array_equal(bypass_scores, bypass_router[:1])
```

OUTPUT

```
Router, up, and down shapes (4, 4) (4, 4, 8) (4, 8, 4)
Input row           [[1. 0. 0. 0.]]
Computed router row [[-2.302585093 -0.510825624 -1.609437912 -2.302585093]]
First row of router [[-2.302585093 -0.510825624 -1.609437912 -2.302585093]]
```

```
expected_branch = {1: [[1.2, -0.6, 0., 1.8]], 0: [[0., 0., 0., 0.]]}
expected_residual = {1: [[2.2, -0.6, 0., 1.8]], 0: [[1., 0., 0., 0.]]}

for capacity in (1, 0):
    _, ids, gates, accepted = route_top1(
        bypass_scores, capacity=capacity
    )
    update = expert_branch(
        bypass_input, ids, gates, accepted, bypass_up, bypass_down
    )
    residual = bypass_input + update

    np.testing.assert_array_equal(ids, [1])
    np.testing.assert_array_equal(accepted, [capacity == 1])
    np.testing.assert_allclose(gates, [0.6], rtol=0, atol=1e-12)
    np.testing.assert_allclose(
        update, expected_branch[capacity], rtol=0, atol=1e-12
    )
    np.testing.assert_allclose(
        residual, expected_residual[capacity], rtol=0, atol=1e-12
    )
    if capacity == 0:
        np.testing.assert_array_equal(residual, bypass_input)

    print(f"Capacity {capacity}")
    print("  Selected ID, gate, accepted", ids, gates, accepted)
    print("  Expert branch update       ", update)
    print("  After adding input         ", residual)
```

OUTPUT

```
Capacity 1
  Selected ID, gate, accepted [1] [0.6] [ True]
  Expert branch update        [[ 1.2 -0.6  0.   1.8]]
  After adding input          [[ 2.2 -0.6  0.   1.8]]
Capacity 0
  Selected ID, gate, accepted [1] [0.6] [False]
  Expert branch update        [[0. 0. 0. 0.]]
  After adding input          [[1. 0. 0. 0.]]
```

The branch update and residual output answer different questions. At capacity zero, the expert branch changes no features. The token's input still travels through the residual path, which is why its output is the original vector. The acceptance mask is the part of the router result that changes between these runs. The next exercise compares requests and acceptances for a whole batch.

9. Compare requests with probabilities

Suppose five of eight tokens ask for expert 2. That expert can receive three of them at capacity three or two of them at capacity two. We still want to describe how many tokens asked for it and how strongly the router favored it. Those are two different summaries of the same router scores.

A row of the identity matrix contains a single 1 and three zeros. Multiplying it by the router matrix selects the row at the position of that 1. The index list `[1, 0, 0, 0, 0, 0, 2, 3]` selects router row 0 five times and each of the other rows once. The teaching weights are logarithms of probability vectors. Exponentiating and normalizing a selected row recovers its original probabilities because they already sum to one.

For each expert i, compute the **request fraction** f_i by counting how many rows chose it and dividing by eight. The **mean probability** P_i averages column i over all eight rows, including rows that chose another expert. For example, five rows give expert 2 probability 0.6 and the three others give it 0.2 each, so $P_2 = (5 \cdot 0.6 + 3 \cdot 0.2)/8 = 0.45$. Its request fraction is $f_2 = 5/8 = 0.625$.

```
from examples.ch09_scaling.model import route_top1
from examples.ch09_scaling.run_examples import teaching_weights

balance_x = np.eye(4, dtype=np.float64)[[1, 0, 0, 0, 0, 0, 2, 3]]
balance_router, _, _ = teaching_weights()
balance_scores = balance_x @ balance_router
print("Input rows (one feature active per row)")
print(balance_x)

balance_probs, balance_ids, balance_gates, mask3 = route_top1(
    balance_scores, capacity=3
)
print("Router probability rows")
print(balance_probs)
print("Requests in one-based expert numbering:", balance_ids + 1)
```

OUTPUT

```
Input rows (one feature active per row)
[[0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
Router probability rows
[[0.6 0.2 0.1 0.1]
 [0.1 0.6 0.2 0.1]
 [0.1 0.6 0.2 0.1]
 [0.1 0.6 0.2 0.1]
 [0.1 0.6 0.2 0.1]
 [0.1 0.6 0.2 0.1]
 [0.1 0.2 0.6 0.1]
 [0.1 0.2 0.1 0.6]]
Requests in one-based expert numbering: [1 2 2 2 2 2 3 4]
```

NumPy counts the returned expert IDs from 0 through 3. The printed requests add one so that they use the book's expert numbers. Use `np.bincount` on the full array of IDs for the request counts and on just the IDs with a true mask entry for accepted counts. Setting `minlength=4` keeps a column even if it gets no rows. Use `mean(axis=0)` to average the probability rows while preserving one result per expert.

The chapter's raw balance term multiplies the two summaries expert by expert and adds the contributions, then multiplies by the number of experts $E = 4$. Here the four products are 0.0203125, 0.28125, 0.028125, and 0.0203125.

$$b = E \sum_{i=1}^{E} f_i P_i = 4(0.0203125 + 0.28125 + 0.028125 + 0.0203125) = 1.4.$$

Expert 2 contributes most because it has both the largest fraction of requests and the largest mean probability. Multiplying b by a training coefficient would give its weighted contribution to the training loss. We report b itself here. Repeat the calculation for capacity two, then compare the returned arrays. Use float64 with an absolute tolerance of 10^{-12} and zero relative tolerance for the probability calculations. Check integer counts and masks exactly.

```
probability2, ids2, gates2, mask2 = route_top1(
    balance_scores, capacity=2
)
np.testing.assert_array_equal(balance_ids, ids2)
np.testing.assert_allclose(
    balance_probs, probability2, atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    balance_gates, gates2, atol=1e-12, rtol=0
)

for cap, probs, ids, accepted in (
    (3, balance_probs, balance_ids, mask3),
    (2, probability2, ids2, mask2),
):
    expert_count = probs.shape[1]
    requests = np.bincount(ids, minlength=expert_count)
    processed = np.bincount(ids[accepted], minlength=expert_count)
    f = requests / len(ids)
    P = probs.mean(axis=0)
    raw_balance = expert_count * (f @ P)
    overflow_rows = np.flatnonzero(~accepted) + 1

    np.testing.assert_array_equal(requests, [1, 5, 1, 1])
    np.testing.assert_array_equal(processed, [1, cap, 1, 1])
    np.testing.assert_allclose(
        f, [1/8, 5/8, 1/8, 1/8], atol=1e-12, rtol=0
    )
    np.testing.assert_allclose(
        P, [0.1625, 0.45, 0.225, 0.1625], atol=1e-12, rtol=0
    )
    np.testing.assert_allclose(raw_balance, 1.4, atol=1e-12, rtol=0)

    print(f"Capacity {cap}")
    print("  Requested / expert", requests.tolist())
    print("  Accepted / expert ", processed.tolist())
    print("  Accepted total    ", accepted.sum())
    print("  Overflow rows     ", overflow_rows.tolist())
    print("  f                 ", np.round(f, 4))
    print("  P                 ", np.round(P, 4))
    print(f"  Raw balance term   {raw_balance:.6f}")
```

```
OUTPUT
Capacity 3
  Requested / expert [1, 5, 1, 1]
  Accepted / expert  [1, 3, 1, 1]
  Accepted total     6
  Overflow rows      [5, 6]
  f                  [0.125 0.625 0.125 0.125]
  P                  [0.1625 0.45   0.225  0.1625]
  Raw balance term   1.400000
Capacity 2
  Requested / expert [1, 5, 1, 1]
  Accepted / expert  [1, 2, 1, 1]
  Accepted total     5
  Overflow rows      [4, 5, 6]
  f                  [0.125 0.625 0.125 0.125]
  P                  [0.1625 0.45   0.225  0.1625]
  Raw balance term   1.400000
```

Capacity three accepts rows 2, 3, and 4 for expert 2. With capacity two, row 4 also overflows. The other experts each process their single request in both runs, so total acceptance changes from six rows to five. The probabilities, selected IDs, gates, f, P, and b agree because the router scores are the same and capacity is applied after the requests are made. A gate is returned even for a rejected request. The expert branch uses the acceptance mask to decide whether to apply it.

If we formed f from processed rows, changing capacity would change our estimate of demand even with the same input and weights. Dividing the processed counts by eight at capacity three would also give fractions that sum to 6/8. The chapter's definition counts every request before filtering, so its fractions sum to one at both capacities.

11. Find the best continuous allocation (optional)

The 13-billion result in Exercise 5 came from a short list of sizes. Suppose we can choose any positive model size and can use the corresponding token count as a real number. This gives a continuous problem. At first, the budget fixes $Q = N_B D_B = 4000$. The token count must be $D_B = Q/N_B$.

Write $z = \sqrt{N_B} > 0$. Then $N_B = z^2$ and $\sqrt{D_B} = \sqrt{Q}\ /z$. The positive square root is important because z represents the square root of a positive count. After substitution, the two penalties and the floor give

$$\ell_{toy} = 1 + \frac{0.4}{z} + \frac{2z}{\sqrt{Q}} = 1 + \frac{a}{z} + bz, \qquad a = 0.4, \quad b = \frac{2}{\sqrt{Q}}.$$

As the model grows, a/z falls and bz rises. Their product $(a/z)(bz) = ab$ stays fixed. The arithmetic–geometric mean inequality says the sum of two positive numbers is at least twice the square root of their product. We can see it here by writing the excess above that lower bound as a square.

$$\frac{a}{z} + bz - 2\sqrt{ab} = \left(\sqrt{a/z} - \sqrt{bz}\right)^2 \geq 0.$$

The lower bound is attained when the square is zero. This requires $a/z = bz$, so **the two penalties are equal at the minimum**. Since $a, b > 0$, exactly one positive z satisfies that condition. Solving for z^2 then gives the size and token count in billions.

$$N_B^* = z^2 = \frac{a}{b} = 0.2\sqrt{Q}\ , \qquad D_B^* = \frac{Q}{N_B^*} = 5\sqrt{Q}\ .$$

At $Q = 4000$, these counts are approximately 12.649111 billion parameters and 316.227766 billion token presentations. Each penalty is about 0.112468265, so the total including the floor is about 1.224936530.

The compute budget is proportional to Q because $C \approx 6 \cdot 10^{18} Q$. Quadrupling C from $2.4 \cdot 10^{22}$ to $9.6 \cdot 10^{22}$ makes Q four times larger. Both optimal counts are proportional to $\sqrt{Q}$, so both double. The new continuous optimum is about 25.298221 billion parameters and 632.455532 billion token presentations. The ratio $D_B^*/N_B^* = 25$ is constant in this example.

To check the predictions, run both sweeps from 5.0 through 40.0 billion. We generate integer tenths 50 through 400 and divide each by 10. This includes the two requested endpoints and avoids accumulating a floating-point increment 350 times. The code computes the continuous result from the budget, checks equality of the two penalties, and then finds the best row on the grid. We also print the neighbors on each side because their losses can look equal when rounded to only five decimals.

```
fine_sizes = [tenths / 10 for tenths in range(50, 401)]
assert len(fine_sizes) == 351
assert (fine_sizes[0], fine_sizes[-1]) == (5.0, 40.0)

continuous_sizes = []
expected_grid_sizes = (12.6, 25.3)

for factor, expected_size in zip((1, 4), expected_grid_sizes):
    budget = factor * sweep_budget
    q = budget / (6 * BILLION**2)
    n_star = 0.2 * np.sqrt(q)
    d_star = q / n_star
    size_penalty = 0.4 / np.sqrt(n_star)
    token_penalty = 2 / np.sqrt(d_star)
    loss_star = 1 + size_penalty + token_penalty
    continuous_sizes.append((n_star, d_star))

    np.testing.assert_allclose(
        size_penalty, token_penalty, rtol=1e-14, atol=0,
    )

    fine_rows = allocation_sweep(budget, fine_sizes)
    best_index = min(
        range(len(fine_rows)), key=lambda i: fine_rows[i][2]
    )
    best_size, best_data, best_loss = fine_rows[best_index]
    assert best_size == expected_size
    np.testing.assert_allclose(
        best_size * best_data, q, rtol=1e-14, atol=0,
    )
    gap = best_loss - loss_star
    assert gap >= 0

    print(f"Budget {budget:.1e} FLOPs (Q={q:g})")
    print(f"Continuous N_B={n_star:.9f}, D_B={d_star:.9f}")
    print(f"Continuous loss={loss_star:.12f}")
    print("      N_B          D_B         Grid loss")
    for i in range(best_index - 1, best_index + 2):
        n_b, d_b, loss = fine_rows[i]
        mark = "  <- best" if i == best_index else ""
        print(f"    {n_b:5.1f}    {d_b:11.6f}    {loss:.12f}{mark}")
    print(f"Best grid loss minus continuous loss: {gap:.3e}")
    print()

np.testing.assert_allclose(
    continuous_sizes[1],
    2 * np.asarray(continuous_sizes[0]),
    rtol=1e-14, atol=0,
)
```

```
OUTPUT
Budget 2.4e+22 FLOPs (Q=4000)
Continuous N_B=12.649110641, D_B=316.227766017
Continuous loss=1.224936530076
      N_B          D_B         Grid loss
     12.5     320.000000    1.224940483865
     12.6     317.460317    1.224936955567  <- best
     12.7     314.960630    1.224936983349
Best grid loss minus continuous loss: 4.255e-07

Budget 9.6e+22 FLOPs (Q=16000)
Continuous N_B=25.298221281, D_B=632.455532034
Continuous loss=1.159054145753
      N_B          D_B         Grid loss
     25.2     634.920635    1.159054446621
     25.3     632.411067    1.159054145852  <- best
     25.4     629.921260    1.159054466266
Best grid loss minus continuous loss: 9.828e-11
```

At the original budget the grid winner is 12.6 billion parameters and about 317.460317 billion token presentations. The two nearby sizes 12.6 and 12.7 bracket the continuous answer of about 12.649111. The 12.6 row has the lower loss, but the difference between those two rows is only about $2.78 \cdot 10^{-8}$. Comparing the returned values before rounding preserves that distinction.

At four times the budget the winner is 25.3 billion parameters and about 632.411067 billion tokens. Here the continuous size is already very close to 25.3, so the grid's excess loss is about $9.83 \cdot 10^{-11}$. The continuous counts double exactly in the algebra. The grid winners are only approximately doubled because each search restricts the size to a grid point and then recomputes the token count from the budget. Twice 12.6 is 25.2, but 25.3 gives a lower loss at the larger budget.

The equal-penalty rule and the 25-to-1 ratio follow from this particular toy loss with two inverse square-root penalties and the coefficients 0.4 and 2. Applying the method to real training would require measuring a loss surface on held-out text and estimating compute for the models being compared.